AI 协作反思日志 — Phase 2 体系结构设计

团队：辛巴巴把鹿比啦 日期：2026.05.02

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
架构风格初选	Claude / Codex	直接询问项目应采用什么架构	3	65%
约束条件下的架构建议	Claude / Codex	加入团队规模、周期、技术背景后重新咨询架构方案	4	55%
候选架构对比	Claude / Codex	要求对比分层、MVC、微服务等方案并给出结论	4	50%
架构辩论整理	Codex	将不同 Prompt 与 AI 回答整理为辩论记录	4	45%
模块划分与职责边界	Codex	基于核心流程划分模块、优先级和边界	4	60%
架构设计文档与 ADR	Codex	生成架构图、技术选型说明、ADR 初稿	4	55%

2. AI 助力分析（Q1）

本阶段 AI 最大的贡献是：快速给出多种可比较的架构候选方案，并帮助团队把“架构风格、模块划分、技术选型、ADR”这些原本分散的内容整理成可讨论的初稿。AI 的作用主要是提供备选视角和初始材料，最终的方案筛选、模块边界修正、功能优先级判断和文档定稿主要由团队完成。

团队人工完成的主要工作包括：

- 结合 P1 阶段问卷、访谈和需求规格说明，重新确认推荐系统是核心功能，而不是后续扩展功能。
- 根据课程周期、团队人数和当前代码成熟度，否定了 AI 初始推荐的微服务方案。
- 重新划分模块职责，把任务分类、任务筛选和管理员任务审核纳入核心模块。
- 明确平台采用自有账号体系，并选择 Gitea 作为外部代码托管底座。
- 对 AI 生成的 ADR 初稿逐条检查，补充了质量属性、实现风险和未采纳方案说明。

不同 Prompt 策略下 AI 回答质量的差异：

- 直接问“推荐什么架构”时，AI 容易给出理想化、高配化答案，偏向微服务和事件总线，需要团队人工筛掉不符合课程约束的内容。
- 在补充团队规模、周期与技术背景后，AI 回答明显更务实，但仍需要人工结合现有仓库依赖、成员经验和功能优先级继续修正。
- 当要求 AI 做候选方案对比时，输出最有利于团队讨论，因为它能提供对比维度；但每个维度的权重仍由团队决定。
- 让 AI 扮演“架构师”时，最容易得到关于过度设计、技术栈先行等风险提醒，但具体风险是否成立仍需要团队结合项目实际判断。

3. AI 误导分析（Q2）

AI 在架构建议中的典型问题：

- 问题1：在约束不充分时，AI 倾向推荐事件驱动微服务。如何发现：与课程总工时、团队人数和当前仓库实现状态对照后发现明显不现实。如何修正：补充“4 人团队、10 周、当前仅脚手架”的约束，并人工收敛到模块化单体。
- 问题2：AI 曾将推荐系统描述为后续扩展能力。如何发现：与问卷、访谈和团队讨论结论对照后，发现推荐是核心功能而非附加项。如何修正：统一修改 P2 文档，把推荐系统提升为 P0 核心模块。
- 问题3：AI 对外部平台的建议混杂了 GitHub OAuth、GitHub API、GitLab 和 Gitea，没有清晰区分“登录方式”和“代码托管底座”。如何发现：团队讨论产品边界时发现平台并不需要依附 GitHub。如何修正：明确采用平台自有账号体系，并统一选用 Gitea 作为外部代码托管底座。

4. Prompt 改进计划（Q3）

上阶段（P1）提出的改进措施，验证结果：

- “证据约束 + 范围约束 + 反例约束”的 Prompt 模板是有效的
- 只要明确团队规模、课程工期和实现边界，AI 的输出会明显更接近真实工程可行方案
- 如果不提供约束，AI 仍然会默认按“理想商业系统”思路输出
- 仅依赖 Prompt 仍然不够，团队必须对 AI 输出进行人工审查，尤其是核心功能优先级和平台边界问题

本阶段新的改进措施：

- 在提问架构问题时，必须同时给出团队规模、开发周期、现有代码成熟度和核心功能优先级
- 对 AI 的架构建议增加“为什么不选其他方案”的追问，避免只给单一结论
- 对涉及平台边界的问题，明确区分“登录方式”“外部依赖”“核心业务模块”三种层次，防止 AI 混淆

5. 本阶段的核心工程判断

决策内容：采用“前后端分离 + 模块化单体 + Gitea 外部代码托管底座 + 平台自有账号体系”的总体架构，并将推荐系统纳入核心业务闭环。

为什么 AI 无法做这个决策：这个判断依赖团队对课程目标、可交付性、实现能力和功能优先级的综合权衡。AI 可以给出大量候选方案，但无法替代团队根据现实工时、演示目标和成员经验做最后取舍。

本阶段团队最终人工判断包括：

- 不采用微服务架构，因为它会显著增加部署、联调和分布式一致性成本。
- 不依赖 GitHub OAuth 作为平台账号基础，因为项目定位是独立平台，而不是 GitHub 的附属工具。
- 不自研完整代码托管底座，因为仓库、Issue、PR 和 Webhook 能力可以由 Gitea 复用。
- 不把推荐系统后置，因为它直接影响初学者能否找到合适任务，是产品闭环的一部分。
- 不单独拆出管理员模块，而是把管理员审核和治理能力落到用户权限、任务管理和审核反馈模块中。